

Day 7 Report – VM, Docker, Volume Persistence & Azure Container Registry

Overview

On Day 7 of my internship, I worked on cloud infrastructure and containerization tasks. The work included creating an Ubuntu Server virtual machine, deploying Docker containers, configuring networking access, implementing persistent storage using Docker volumes, and pushing container images to Azure Container Registry (ACR) as a private registry.

Task 1: VM Creation and Docker Container Deployment

Objective

- Create Ubuntu Server Virtual Machine
 - Install Docker Engine
 - Run Docker container from image
 - Expose container using VM Public IP
-

Step 1: Create Ubuntu Server VM

- Created a Virtual Machine using Ubuntu Server OS
- Configured:
 - Public IP enabled
 - Required ports allowed in security group / firewall

Screenshot: VM creation summary
(Attach Screenshot 1 here)

Step 2: Install Docker on VM

Updated packages and installed Docker.

Commands Used

```
sudo apt update
sudo apt install docker.io -y
sudo systemctl start docker
sudo systemctl enable docker
```

`docker --version`

Screenshot: Docker installation and version output
(*Attach Screenshot 2 here*)

Step 3: Pull Docker Image

Pulled required image from Docker Hub (example app running on port 80 / 8080).

Command

`docker pull <image-name>`

Screenshot: Docker image pull success
(*Attach Screenshot 3 here*)

Step 4: Run Docker Container

Mapped container port to VM port.

Command Example

`docker run -d -p 8080:80 <image-name>`

Explanation:

- 8080 → VM public port
- 80 → Container internal port

Screenshot: Running container list
(*Attach Screenshot 4 here*)

Step 5: Access Application via Public IP

Accessed container application using:

`http://<VM-Public-IP>:8080`

Screenshot: Application opened in browser
(*Attach Screenshot 5 here*)

Task 2: Docker Volume Creation & Data Persistence

Objective

Ensure data remains available even after container deletion.

Step 1: Create Docker Volume

```
docker volume create myvolume
```

Screenshot: Volume creation confirmation
(Attach Screenshot 6 here)

Step 2: Run Container with Volume Mapping

```
docker run -d -p 8080:80 -v myvolume:/app/data <image-name>
```

Screenshot: Container running with volume
(Attach Screenshot 7 here)

Step 3: Upload File Inside Container

Uploaded or created test file inside container storage path.

Screenshot: File present inside container
(Attach Screenshot 8 here)

Step 4: Delete Container

```
docker rm -f <container-id>
```

Screenshot: Container deleted
(Attach Screenshot 9 here)

Step 5: Restore Data Using Same Volume

Ran new container using same volume.

```
docker run -d -p 8080:80 -v myvolume:/app/data <image-name>
```

Verified file was still available.

Screenshot: File restored successfully
(*Attach Screenshot 10 here*)

Task 3: Push Container Image to Azure Container Registry (ACR)

Objective

- Create private container registry in Azure
 - Push container image to ACR
 - Access image from Azure portal
-

Step 1: Install Azure CLI on VM

```
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
```

Screenshot: Azure CLI installation
(*Attach Screenshot 11 here*)

Step 2: Login to Azure Account

```
az login
```

Authenticated using browser or token.

Screenshot: Azure login success
(*Attach Screenshot 12 here*)

Step 3: Login to Azure Container Registry

```
az acr login --name <acr-name>
```

Screenshot: ACR login success
(Attach Screenshot 13 here)

Step 4: Tag Docker Image for ACR

`docker tag <image-name> <acr-name>.azurecr.io/<image-name>:v1`

Screenshot: Image tagging output
(Attach Screenshot 14 here)

Step 5: Push Image to ACR

`docker push <acr-name>.azurecr.io/<image-name>:v1`

Screenshot: Image push success
(Attach Screenshot 15 here)

Step 6: Verify in Azure Portal

Checked repository in Azure Container Registry console.

Screenshot: Image visible in Azure Portal
(Attach Screenshot 16 here)

Outcome / Learning

Through this task, I learned:

- Virtual Machine provisioning
 - Docker container lifecycle management
 - Port mapping and public access
 - Persistent storage using Docker volumes
 - Container image management
 - Working with Azure CLI
 - Using private container registries
-

Conclusion

Successfully deployed containerized application on Ubuntu VM, implemented persistent storage using Docker volumes, and uploaded container image to Azure Container Registry as a private image repository.
